QuestLog

Od pull requesta do produkcji bez logowania na serwer.

L5. Proces CI/CD

GitHub Actions · self-hosted runnery na OCI · questlog.com.pl

CO BYŁO DO ZROBIENIA

Wymagania zadania

Projekt procesu CI/CD

Jeden pipeline w GitHub Actions: CI na każdym PR, CD po merge'u do main

Automatyczna weryfikacja

Testy API na PostgreSQL 16, build frontu, smoke test całego stacku w Dockerze

Automatyczne wdrożenie

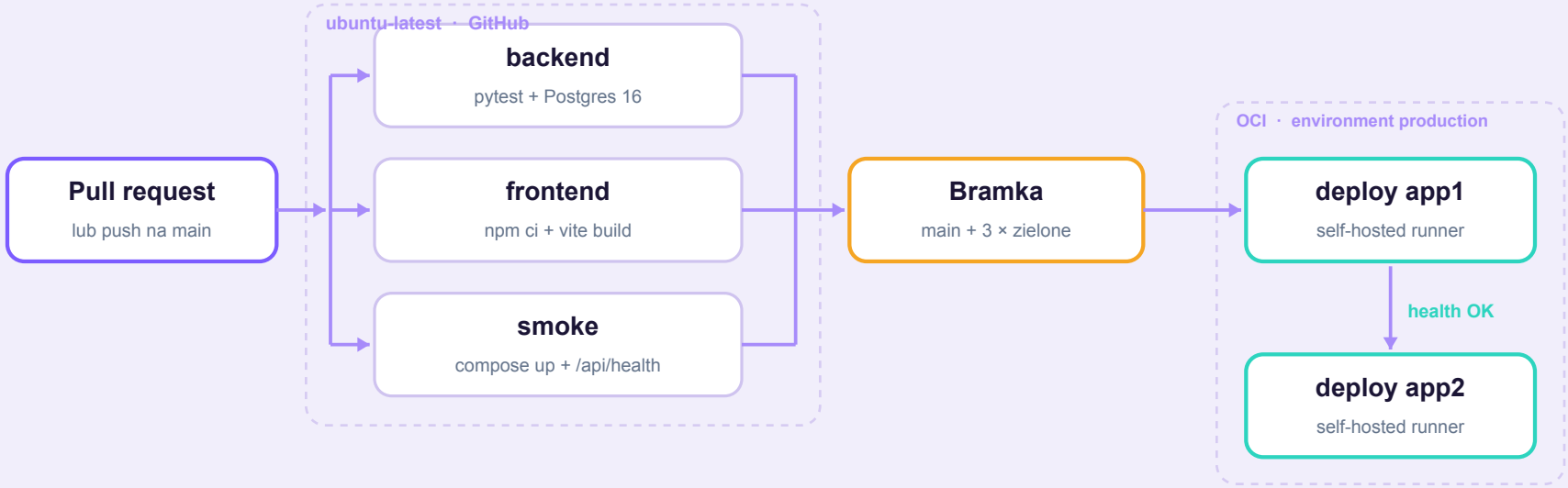
Rolling deploy na app1 i app2 z health checkiem po każdej maszynie

Prezentacja implementacji

Ten deck oraz przebieg na żywo w zakładce Actions repozytorium

.GITHUB/WORKFLOWS/CI-CD.YML

Pipeline



- Na pull requeście działa tylko CI; job deploy jest pomijany, a ruleset blokuje merge przy czerwonym checku.
- Rollback to git revert na main — pipeline sam wdroży poprzednią wersję.

CO SPRAWDZAMY NA KAŻDYM PR

Ciągła integracja

backend

11 testów API na prawdziwym PostgreSQL 16 uruchomionym jako kontener usługi.

frontend

npm ci + vite build: zależności z lockfile'a i kompilacja całej aplikacji React.

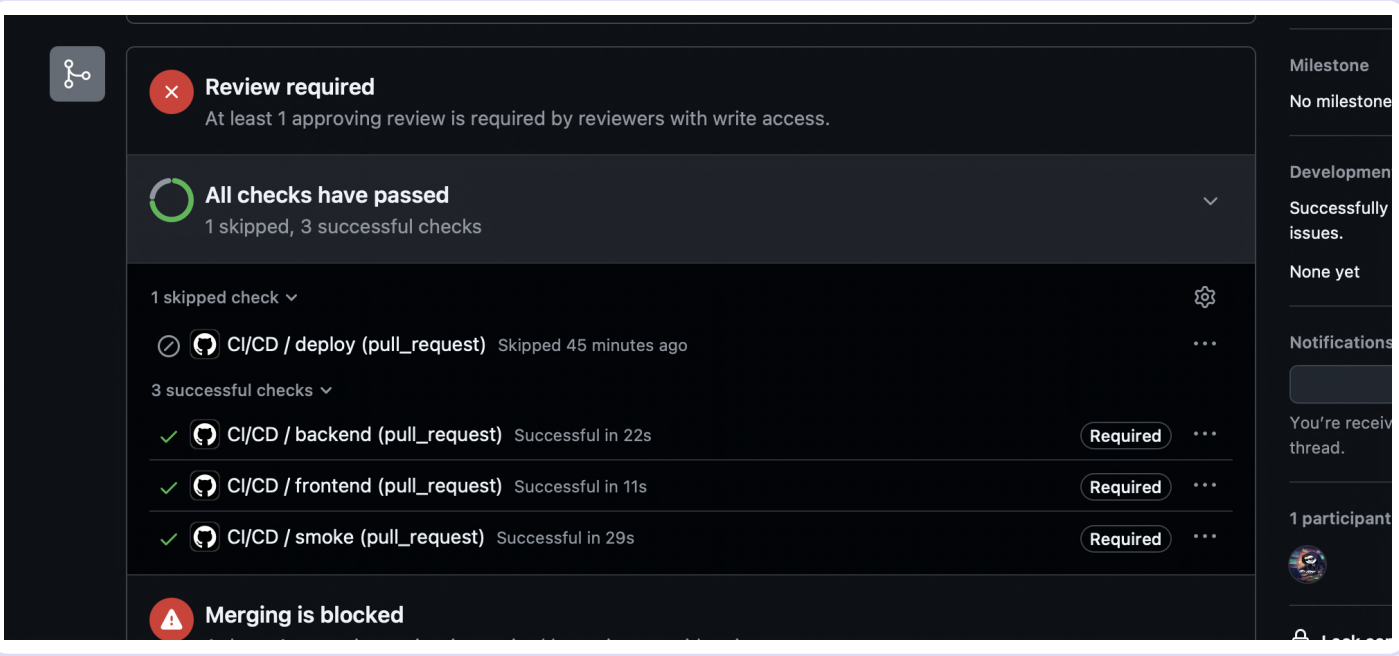
smoke

docker compose up --wait całego stacku i zapytanie przez Caddy do /api/health.

- Joby idą równolegle na maszynach GitHuba; cache pip i npm skraca kolejne przebiegi.
- Nowy commit w PR anuluje poprzedni przebieg (concurrency), więc nie czekamy na nieaktualne wyniki.
- Smoke test buduje te same Dockerfile'e, które trafiają na produkcję — błąd w obrazie wychodzi przed merge'em.

PULL REQUEST #7 · WYMAGANE CHECKI ZIELONE

Pierwszy przebieg na GitHubie



backend 22 s · frontend 11 s · smoke 29 s · deploy pominięty na PR-ze, bo działa tylko po merge'u do main

DLACZEGO SELF-HOSTED RUNNER

Ciągłe wdrażanie

SSH z GitHuba

- port 22 otwarty dla puli adresów GitHuba
- klucz SSH jako dodatkowy sekret do utrzymania
- łamie regułę nsg-app z L3

Runner na VM

- łączy się z GitHubem wychodząco (443)
- żadnych nowych reguł w sieci
- sekrety z GitHuba zapisywane do .env przy deployu

- Każda maszyna ma własny runner z etykietą app1 lub app2 — job wskazuje maszynę etykietą.
- Runner działa jako usługa systemd na koncie gha w grupie docker, bez uprawnień roota.
- Logi wdrożenia, czas i wersja widoczne w GitHubie, bez logowania się na serwer.

JEDNA MASZYNA NARAZ, DRUGA TRZYMA RUCH

Rolling deploy

git reset --hard

dokładnie ten commit, który przeszedł CI

→

compose up --build

przebudowa obrazów na maszynie

→

curl /api/health

do 30 prób co 2 s

→

następna maszyna

dopiero po zielonym health checku

- Gdy app1 restartuje kontenery, health check load balancera kieruje cały ruch na app2.
- Padnie app1 — app2 w ogóle nie startuje i dalej serwuje starą, działającą wersję.
- Wdrożenia na tę samą maszynę nie nakładają się: kolejny merge czeka w kolejce.
- Schemat bazy tworzy się idempotentnie przy starcie backendu, więc deploy nie wymaga ręcznych migracji.

KTO I CO MOŻE WDROŻYĆ

Zabezpieczenia

Ruleset na main

zmiany tylko przez PR, merge wymaga zielonych backend, frontend i smoke

Environment production

wdrożenie możliwe wyłącznie z gałęzi main

PR z forków

wymagają zatwierdzenia — obcy kod nie trafi na nasze runnery

Minimalne uprawnienia

token workflow tylko do odczytu; sekrety tylko w environment production

Dziękuję za uwagę

github.com/frodz1/webowka/actions

QuestLog · L5. Proces CI/CD